

In []:

Machine Learning-Based Salary Prediction and Analysis using NYC Job Postings Data

TABLE OF CONTENTS – Salary Prediction Project

Step 1: Executive Summary

Step 2: Problem Statement & Objectives

Step 3: Dataset Overview

Step 4: Methodology

Overview of data processing and analysis approach

Step 4.1: Data Preprocessing

- **Step4.1.1** Missing Value Handling
- **Step4.1.2** Dropping Irrelevant Columns
- **Step4.1.3** Data Cleaning

Step 4.2: Feature Engineering

- **Step 4.2.1:** Average Salary Creation
- **Step 4.2.2:** Posting Year & Month Extraction
- **Step 4.4.3:** Feature Optimization

Step 4.3: Exploratory Data Analysis

- **Step 4.3.1:** Univariate Analysis
- **Step 4.3.2:** Bivariate Analysis
- **Step 4.3.3:** Correlation Analysis
- **Step 4.3.4:** Temporal Analysis

Step 4.4: Feature Selection

- **Step 4.4.1:** Important Features
- **Step 4.4.2:** Dropped Features

Step 5: Model Development & Training

- **Step 5.1:** Model Selection
- **Step 5.2:** Model Training Approach & Evaluation Metrics

Step 6: Model Comparison & Final Model Selection

Step 7: Final Model Performance & Deployment

- **Step 7.1:** Model Performance
- **Step 7.2:** Streamlit Application (User Interface)
- **Step 7.3:** Dashboard Visualization

Step 8: Business Insights & Recommendations**Step 9: Model Limitations****Step 10: Future Improvements****Step 11: Conclusion****Step 1: Executive Summary**

This project focuses on building a machine learning-based system to predict job salaries using real-world job postings data. In today's competitive job market, understanding salary trends and the factors influencing compensation is crucial for both job seekers and organizations. The primary goal of this project is to analyze job-related attributes and accurately estimate the expected salary based on those features.

The dataset used for this analysis consists of job postings with multiple attributes such as agency, job category, career level, employment type, and location. Initially, the data contained inconsistencies, missing values, and irrelevant information, which required thorough preprocessing. Key steps included handling missing values, removing unnecessary columns, and transforming the data into a structured format suitable for analysis.

To enhance the predictive capability of the model, new features such as average salary and temporal attributes (posting year and month) were engineered. These features helped in capturing trends and patterns over time, improving the overall understanding of salary variations.

Extensive exploratory data analysis (EDA) was performed to uncover relationships between different variables and the target variable (salary). This included analyzing distributions, identifying patterns across categories, and understanding how different factors influence salary levels. The insights gained from this stage played a crucial role in selecting relevant features for model building.

Multiple machine learning models, including Linear Regression, Decision Tree, Random Forest, and XGBoost, were implemented and evaluated. After comparing their performance using metrics such as MAE, RMSE, and R^2 score, Random Forest was selected as the final model due to its high accuracy and ability to generalize well on unseen data.

To make the model accessible and user-friendly, it was deployed using a Streamlit application. The application allows users to input job-related details and receive real-time

salary predictions through an interactive interface. This demonstrates the practical applicability of the model in real-world scenarios.

Overall, this project not only showcases the complete machine learning pipeline—from data preprocessing to deployment—but also provides meaningful insights into salary trends, making it valuable for decision-making in recruitment and career planning.

Step 2: Problem Statement

In today's job market, salary information plays a crucial role in both job search and hiring decisions. However, determining an appropriate salary for a given job role is often challenging due to the influence of multiple factors such as job category, experience level, employment type, and organizational structure.

Organizations need data-driven insights to offer competitive salaries, while job seekers require reliable estimates to make informed career decisions. Traditional approaches rely heavily on manual benchmarking or limited historical data, which may not accurately capture current market trends.

The dataset used in this project contains detailed job postings with various attributes, but it does not directly provide a clear mechanism to predict salary based on these features. This creates a need for a system that can analyze these attributes and accurately estimate salary ranges.

Therefore, the core problem addressed in this project is:

To build a machine learning model that can predict job salaries based on job-related features and uncover key factors influencing salary variations.

Objectives

To address the above problem, the following objectives were defined:

1. Data Understanding and Preparation

To explore the dataset, understand feature distributions, and handle missing or inconsistent data to ensure quality inputs for modeling.

2. Feature Engineering

To create meaningful features such as average salary and temporal attributes (year and month) that help improve model performance.

3. Exploratory Data Analysis (EDA)

To analyze relationships between different variables and identify patterns that influence salary.

4. Model Development

To build multiple machine learning models and evaluate their performance in predicting salaries.

5. Model Optimization

To improve model performance using techniques such as hyperparameter tuning and feature selection.

6. Deployment

To develop a user-friendly Streamlit application that allows real-time salary prediction based on user inputs.

7. Insight Generation

To extract meaningful business insights from the data that can help in decision-making for both organizations and job seekers.

Step 3: Dataset Overview

The dataset used in this project consists of real-world job postings data, which includes detailed information about various job roles across different departments and categories. The data provides a comprehensive view of job-related attributes that influence salary, making it suitable for building a predictive model.

The dataset contains 5120 records and 30 features, representing different aspects of job postings such as agency, job category, career level, employment type, salary range, and work location. Each record corresponds to a unique job posting.

Key Features in the Dataset

The dataset includes a mix of numerical and categorical variables. Some of the important features are:

- Agency – Represents the department or organization offering the job
- Posting Type – Indicates whether the job is internal or external
- Number of Positions – Total vacancies available for the role
- Job Category – Defines the domain or field of the job
- Career Level – Indicates experience level (Entry, Mid, Senior, Executive)
- Employment Type – Full-time or part-time indicator
- Salary Range From & To – Minimum and maximum salary offered
- Salary Frequency – Specifies whether salary is annual, hourly, etc.
- Work Location – Location where the job is based
- Posting Date – Date when the job was posted

Data Types

The dataset consists of different data types:

- Numerical Features
 - Number of Positions
 - Salary Range From
 - Salary Range To
- Categorical Features
 - Agency
 - Job Category
 - Career Level
 - Employment Type
 - Salary Frequency
- Date Features
 - Posting Date
 - Posting Updated
 - Process Date

Initial Observations

During the initial exploration of the dataset, the following observations were made:

- The dataset contains both structured and unstructured data
- Several columns have missing values, especially in text-based fields
- Some features such as job description and preferred - skills contain large textual information
- Salary is provided as a range, which requires transformation for modeling
- Certain columns have high cardinality, which may impact model performance

Target Variable

The target variable for this project is:

Average Salary

Since the dataset provides salary as a range (minimum and maximum), a new feature called **average salary** was created by taking the mean of these two values. This transformed variable is used as the target for prediction.

Step 4: Methodology

The methodology followed in this project is designed to systematically transform raw job postings data into a structured format suitable for building an accurate salary prediction model. The overall approach includes data preprocessing, feature engineering, exploratory data analysis, and feature selection, ensuring that only relevant and meaningful information is used for model development.

The process begins with cleaning and preparing the dataset, followed by creating new features that enhance predictive power. Extensive analysis is then performed to understand patterns and relationships within the data. Finally, important features are selected based on their impact on the target variable, ensuring optimal model performance.

Overall Approach

The methodology can be summarized in the following steps:

- Data Preprocessing – Cleaning the dataset and handling missing or irrelevant values
- Feature Engineering – Creating new variables to improve model performance
- Exploratory Data Analysis (EDA) – Identifying patterns and relationships
- Feature Selection – Retaining only the most important features

This structured approach ensures that the model is built on high-quality data and meaningful features, leading to better accuracy and generalization.

Key Highlights of Methodology

- Raw data was transformed into a clean and usable format through systematic preprocessing
- New features such as average salary and temporal attributes were created to capture hidden patterns
- Data was analyzed from multiple perspectives using statistical and visual techniques
- Irrelevant and high-noise features were removed to reduce model complexity
- The final dataset was optimized for machine learning model training

Step 4.1: Data Preprocessing

Data preprocessing is a critical step in the machine learning pipeline, as the quality of input data directly impacts the performance of the model. The raw dataset contained missing values, irrelevant columns, inconsistent formats, and high-dimensional text features, which required systematic cleaning and transformation.

The goal of preprocessing was to convert the raw job postings data into a clean, structured, and model-ready dataset.

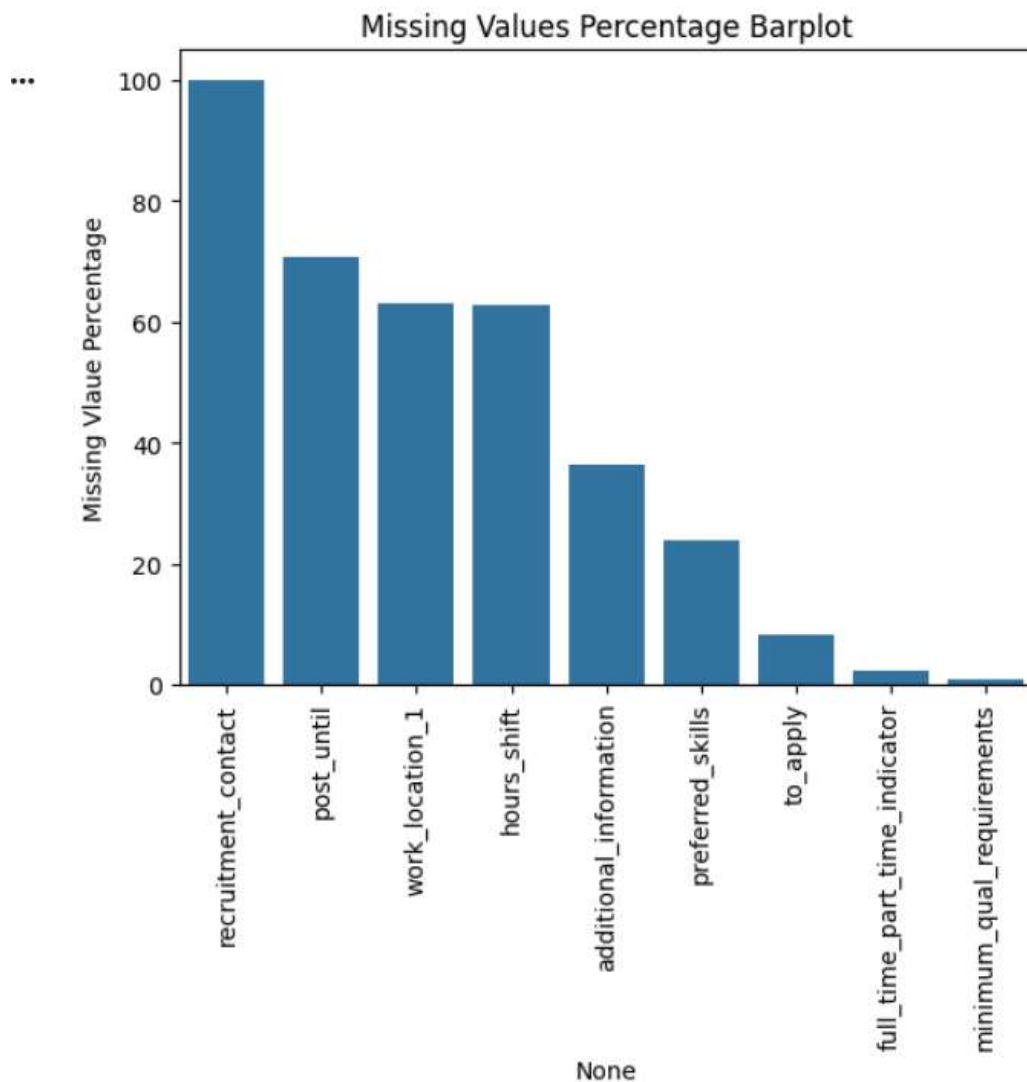
4.1.1 Handling Missing Values

During initial analysis, several columns were found to contain missing values. These were handled based on their importance and the proportion of missing data.

- Columns with very **high missing values** (e.g., recruitment contact, post until, work location 1, hours/shift) were dropped, as they did not provide sufficient information for analysis.

- Columns with **low missing values** were handled using appropriate strategies such as filling with the most frequent value (mode).
- Text-heavy columns with missing values were later removed as part of feature selection.

This approach ensured that the dataset remained consistent without introducing bias.



4.1.2 Dropping Irrelevant Columns

Several columns in the dataset were identified as irrelevant or not useful for predictive modeling. These included:

- Unique identifiers (e.g., job ID)
- High-cardinality text columns (e.g., job description, preferred skills)
- Columns with redundant or duplicate information

Additionally, columns that did not contribute to salary prediction or introduced noise were removed to simplify the dataset and improve model performance.

4.1.3 Data Cleaning

To ensure consistency across the dataset, multiple data cleaning steps were performed:

- Column names were standardized by converting them to lowercase and replacing spaces with underscores
- Data types were verified and corrected where necessary
- Duplicate records were identified and handled appropriately
- Inconsistent formats, especially in date columns, were standardized

These steps helped in creating a uniform structure that could be easily processed during modeling.

4.1.4 Preparation for Modeling

After cleaning and transformation, the dataset was prepared for further analysis and feature engineering. At this stage:

- Only relevant columns were retained
- Missing values were handled
- Data consistency was ensured

This resulted in a refined dataset that served as a strong foundation for subsequent steps such as feature engineering and exploratory data analysis.

Step 4.2: Feature Engineering

Feature engineering plays a crucial role in improving the performance of machine learning models by transforming raw data into meaningful inputs. In this project, new features were created to better capture salary patterns and enhance the predictive capability of the model.

The dataset originally provided salary information as a range and included date-related fields that were not directly usable. These were transformed into structured features to extract more meaningful insights.

4.2.1 Average Salary Creation

The dataset contained two separate columns representing salary:

- Salary Range From
- Salary Range To

Since the objective was to predict a single salary value, a new feature called **average salary** was created by calculating the mean of these two values.

This transformation provided a single, continuous target variable that is more suitable for regression modeling.

📌 Formula used:

$$\text{Average Salary} = (\text{Salary Range From} + \text{Salary Range To}) / 2$$

This step simplified the prediction task and ensured consistency in the target variable.

4.2.2 Temporal Feature Extraction

The dataset included date-related columns such as posting date, which contained valuable time-based information. To utilize this effectively, new features were extracted:

- Posting Year – Represents the year in which the job was posted
- Posting Month – Represents the month of the job posting

These features helped in identifying seasonal trends and changes in salary patterns over time.

By converting raw date values into structured numerical features, the model was able to better understand temporal variations in job postings and salary distributions.

4.2.3 Feature Transformation for Modeling

After creating new features:

- Original salary range columns were no longer required for modeling
- Date columns were replaced with extracted temporal features
- The dataset was transformed into a format suitable for machine learning algorithms

These transformations ensured that the model receives only relevant and informative inputs.

Step 4.3: Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to understand the structure of the dataset, identify patterns, and analyze the relationship between different features and the target variable (average salary). This step played a crucial role in uncovering insights and guiding feature selection for model building.

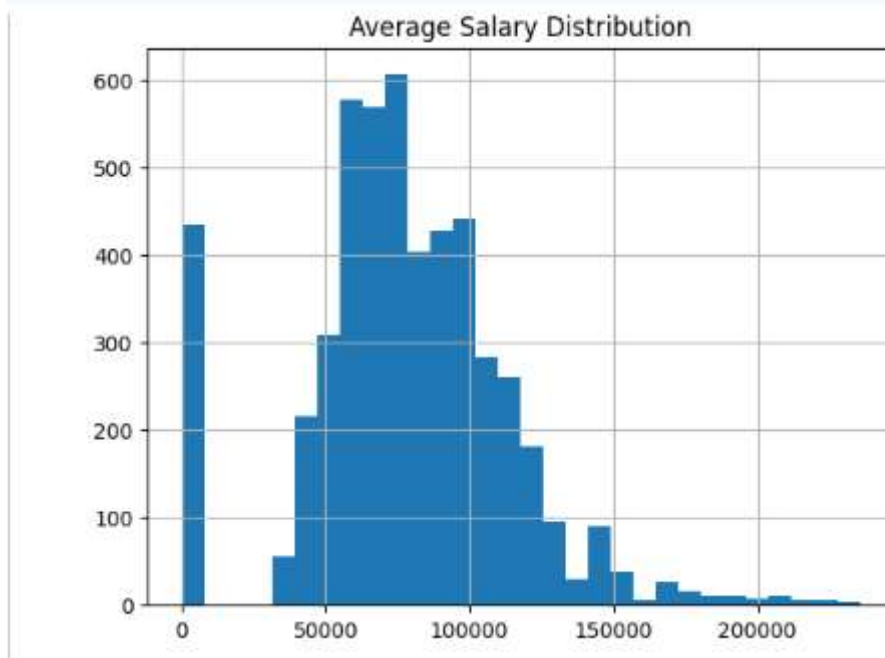
The analysis was carried out using both statistical summaries and visualizations to gain a deeper understanding of the data.

4.3.1 Univariate Analysis

Univariate analysis was performed to examine the distribution of individual features.

Numerical features such as number of positions and average salary were analyzed using summary statistics and distribution plots. Categorical features such as agency, job category, and career level were analyzed using frequency counts.

This helped in identifying skewness, outliers, and dominant categories in the dataset.



4.3.2 Bivariate Analysis

Bivariate analysis was conducted to understand the relationship between features and the target variable.

Numerical features were compared against average salary to observe trends and variations. Categorical features were analyzed using grouped plots to understand how different categories influence salary.

This analysis revealed how factors such as career level, employment type, and job category impact salary.

4.3.3 Correlation Analysis

Correlation analysis was performed on numerical features to identify relationships between variables.

A correlation matrix was used to measure the strength of relationships. Highly correlated features were analyzed to avoid redundancy.

This step helped in understanding dependencies between variables and supported feature selection.

4.3.4 Temporal Analysis

Temporal analysis was performed using posting year and posting month features.

Trends in job postings over time were analyzed Salary variations across different time periods were observed

This helped in identifying seasonal patterns and changes in salary trends.

Step 4.4: Feature Selection

Feature selection is a crucial step in the machine learning pipeline, where only the most relevant features are retained for model training. The objective of this step is to improve model performance, reduce complexity, and eliminate noise from the dataset.

In this project, feature selection was performed based on insights gained from exploratory data analysis, missing value analysis, and feature relevance to the target variable (average salary).

4.4.1 Important Features Selected

Based on analysis, the following features were identified as significant contributors to salary prediction:

- Agency – Different organizations offer varying salary structures
- Posting Type – Internal and external postings show variation in salary
- Number of Positions – Influences hiring demand
- Level – Indicates seniority, which strongly impacts salary
- Job Category – Different domains have different pay scales
- Full-Time/Part-Time Indicator – Employment type affects compensation
- Career Level – One of the most important factors influencing salary
- Salary Frequency – Helps standardize salary representation
- Residency Requirement – May influence eligibility and salary
- Posting Year & Month – Captures temporal trends

These features were retained as they showed meaningful relationships with the target variable and contributed positively to model performance.

4.4.2 Dropped Features

Several features were removed during this stage due to the following reasons:

- High Missing Values
 - Recruitment Contact
 - Post Until
 - Work Location 1
 - Hours/Shift
- High Cardinality / Text-Based Columns
 - Job Description
 - Preferred Skills
 - Additional Information
 - Minimum Qualification Requirements
 - To Apply
- Irrelevant or Redundant Features
 - Job ID (unique identifier)
 - Business Title
 - Civil Service Title
 - Title Classification
- Transformed / Replaced Features
 - Salary Range From & To → replaced by Average Salary
 - Date columns → replaced by Posting Year & Month

4.4.3 Feature Optimization

To further refine the dataset:

- Only features relevant to salary prediction were retained
- Redundant and noisy features were eliminated
- The dataset was simplified without losing important information

This resulted in a clean and optimized feature set that improved both model efficiency and interpretability.

Step 5: Model Development & Training

After completing data preprocessing, feature engineering, and feature selection, the dataset was ready for model development. The objective of this stage was to build machine learning models capable of accurately predicting the average salary based on job-related features.

A structured approach was followed to ensure proper training, evaluation, and comparison of different models.

Step 5.1 : Model Selection

Multiple regression models were selected to evaluate their performance on the dataset. The selection included both simple and advanced algorithms to capture different types of relationships in the data.

The models used in this project are:

- **Linear Regression:** Used as a baseline model to understand the basic linear relationship between features and salary.
- **Decision Tree Regressor:** Captures non-linear patterns and provides easy interpretability.
- **Random Forest Regressor:** An ensemble model that reduces overfitting and improves accuracy.
- **XGBoost Regressor:** A powerful boosting algorithm known for high performance on structured data.

Using multiple models allowed for a comprehensive comparison and helped in identifying the most suitable algorithm.

Step 5.2 : Model Training Approach & Evaluation Metrics

- **Train-Test Split**

The dataset was divided into training and testing sets to evaluate model performance on unseen data.

- 80% of the data was used for training
- 20% of the data was used for testing

This ensured that the model generalizes well and avoids overfitting.

Cross-Validation

To ensure robustness, cross-validation was applied using K-Fold technique. This helped in validating the model across multiple data splits and provided a more reliable estimate of performance.

Evaluation Metrics

The performance of each model was evaluated using the following regression metrics:

- **Mean Absolute Error (MAE):** Measures the average magnitude of errors
- **Root Mean Squared Error (RMSE):** Penalizes larger errors and provides better insight into prediction accuracy
- **R² Score (Coefficient of Determination):** Indicates how well the model explains the variance in the target variable

These metrics provided a comprehensive understanding of model performance.

Step 5.3 : Training Process

Each model was trained using the prepared dataset, and predictions were generated on the test set. The results were then evaluated using the defined metrics.

- Linear Regression provided a baseline performance
- Tree-based models captured complex relationships more effectively
- Ensemble models showed improved accuracy and stability

This systematic training process enabled effective comparison and selection of the best-performing model.

Step 6: Model Comparison & Final Model Selection

After training multiple machine learning models, a comparative analysis was performed to evaluate their performance and identify the most suitable model for salary prediction.

Each model was assessed using standard regression metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R² score. This comparison helped in understanding the strengths and limitations of each model.

** Model Performance Comparison**

The performance of the models is summarized below:

- **Linear Regression** Provided a strong baseline performance, capturing general trends in the data but limited in handling complex non-linear relationships
- **Decision Tree Regressor** Achieved very high accuracy on training data but showed signs of overfitting, making it less reliable for generalization
- **Random Forest Regressor** Delivered the best overall performance with high accuracy and better generalization by reducing overfitting
- **XGBoost Regressor** Performed competitively with slightly lower performance compared to Random Forest but still demonstrated strong predictive capability
- **K-Nearest Neighbors (KNN)** Showed poor performance compared to other models and was not suitable for this dataset

Model	MAE	RMSE	R ² Score
Linear Regression	0.2443	0.3464	0.9760
Decision Tree	0.0590	0.1566	0.9950
Random Forest	0.0844	0.1361	0.9962
XGBoost	0.0932	0.1397	0.9960

Model	MAE	RMSE	R ² Score
KNN	0.2104	1.0309	0.7875

Insight: Among all the models, Random Forest achieved the highest R² score with low RMSE, indicating strong predictive performance and good generalization. Decision Tree showed high accuracy but is prone to overfitting, while KNN performed poorly compared to other models.

Final Model Selection

Based on the comparison, the **Random Forest Regressor** was selected as the final model for this project.

The selection was made considering the following factors:

- Highest R² score indicating strong explanatory power
- Lowest RMSE, reflecting better prediction accuracy
- Ability to handle non-linear relationships effectively
- Reduced overfitting compared to Decision Tree
- Stability across different data splits

6.3 Justification of Final Model

The Random Forest model achieved an R² score of 0.9962, with low RMSE (0.1361) and MAE (0.0844), indicating highly accurate predictions and strong generalization capability.

Cross-validation further confirmed the stability of the model, with consistent R² scores across multiple folds and an average R² score of approximately 0.9967.

This makes it well-suited for the given dataset, where multiple categorical and numerical features influence salary.

Step 7: Final Model Performance & Deployment

After evaluating multiple machine learning models, the Random Forest Regressor was selected as the final model due to its superior performance and ability to generalize well on unseen data. The model was further validated and deployed using a user-friendly interface to enable real-time salary prediction.

7.1 Model Performance

The final model demonstrated excellent performance across all evaluation metrics, indicating high prediction accuracy and robustness.

- **R² Score:** 0.9962
- **RMSE:** 0.1361

- **MAE:** 0.0844

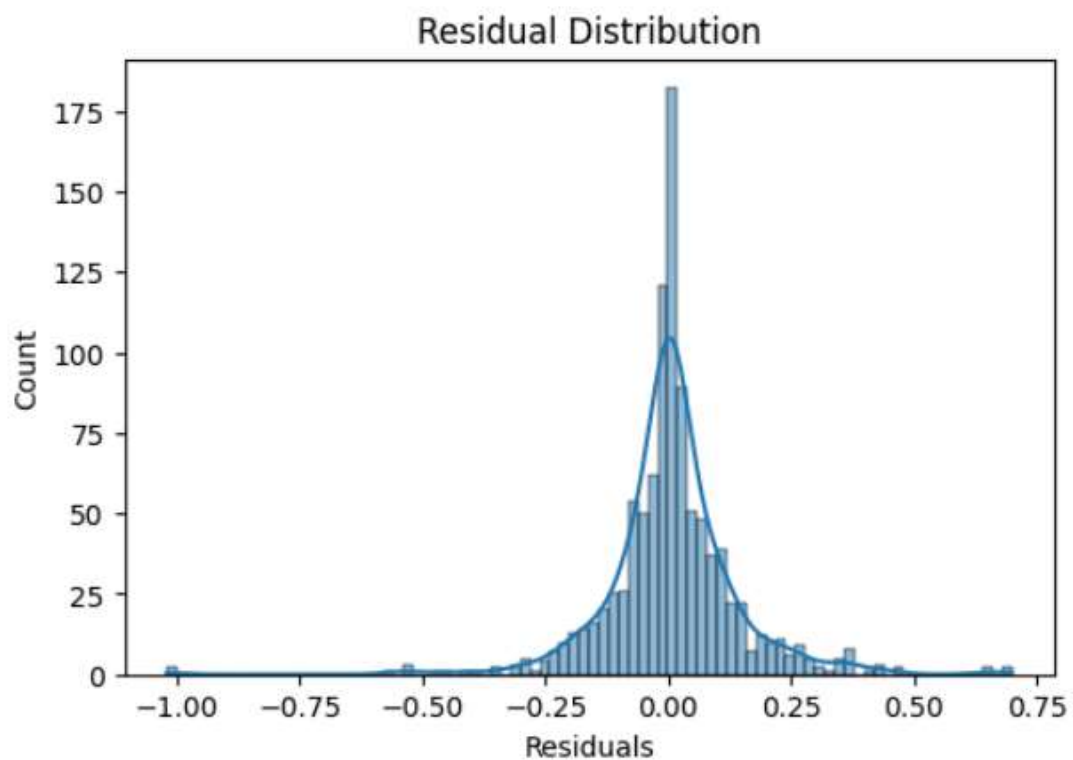
These results show that the model is able to explain a very high proportion of variance in the target variable and produces minimal prediction error.

Additionally, cross-validation was performed to ensure the stability of the model:

- **Cross-validation R^2 Scores:** Consistently above 0.996
- **Average R^2 Score:** ~0.9967

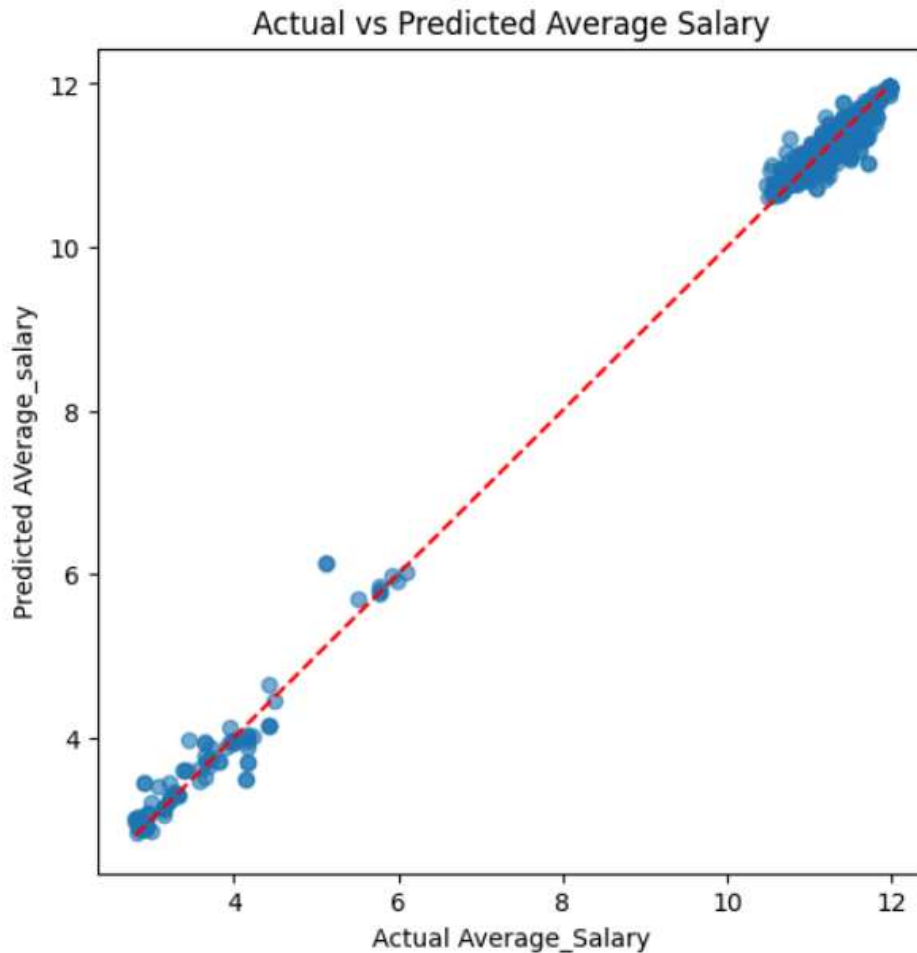
This confirms that the model performs consistently across different data splits and is not overfitting.

 Error Distribution Plot



👉 Insight: The error distribution is centered around zero, indicating that the model predictions are accurate with minimal deviation.

Error Comparison Plot



👉 Insight: Random Forest shows lower error compared to other models, confirming its superior performance.

7.2 Streamlit Application (User Interface)

To make the model accessible and practical for real-world usage, a Streamlit-based web application was developed. This application allows users to input job-related details and receive instant salary predictions.

Application Features

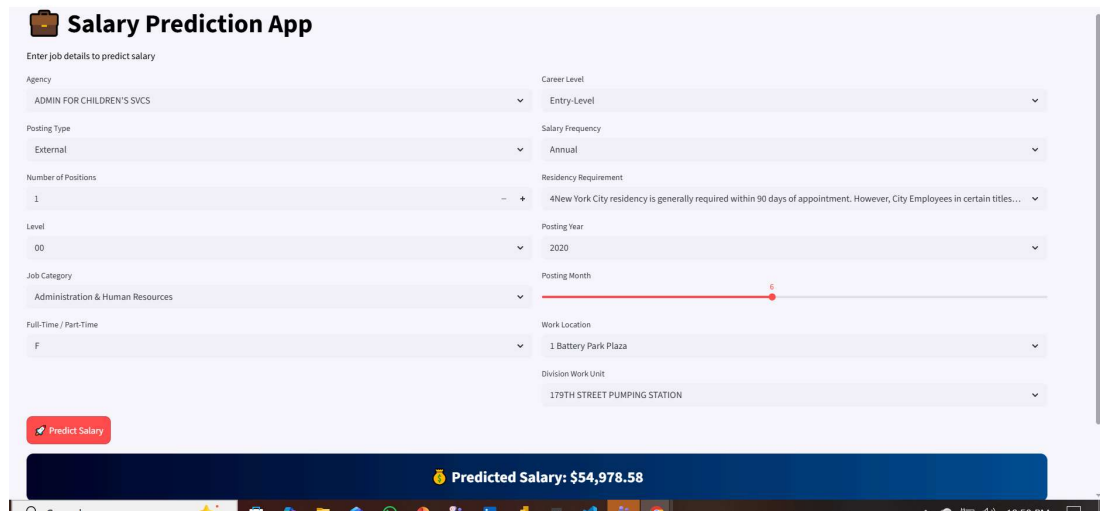
- Interactive and user-friendly interface
- Input fields for job attributes such as:
 - Agency
 - Job Category
 - Career Level
 - Employment Type
 - Location
 - Posting Year & Month

- Real-time salary prediction using the trained model
- Clean and structured layout for better usability

Prediction Workflow

The application follows a structured pipeline:

1. User Input
2. Encoding (using saved encoders)
3. Feature Alignment (matching training columns)
4. Model Prediction
5. Display Predicted Salary



The screenshot displays the 'Salary Prediction App' interface. It features a form with various input fields for job details, organized into two columns. The left column includes fields for Agency (ADMIN FOR CHILDREN'S SVCS), Posting Type (External), Number of Positions (1), Level (00), Job Category (Administration & Human Resources), and Full-Time / Part-Time (F). The right column includes Career Level (Entry-Level), Salary Frequency (Annual), Residency Requirement (4New York City residency is generally required within 90 days of appointment. However, City Employees in certain titles...), Posting Year (2020), Posting Month (a slider set to 6), Work Location (1 Battery Park Plaza), and Division Work Unit (179TH STREET PUMPING STATION). A red 'Predict Salary' button is located at the bottom left of the form. Below the form, a dark blue banner displays the 'Predicted Salary: \$54,978.58' with a gold coin icon.

Technical Implementation

- The trained Random Forest model was saved using joblib
- Encoders used during preprocessing were also saved and reused
- Feature columns were aligned to match the training dataset
- Streamlit was used to build the frontend interface

This ensures consistency between training and prediction phases.

Practical Impact

The deployed application demonstrates how machine learning models can be integrated into real-world systems to support decision-making.

- Helps job seekers estimate expected salary
- Assists organizations in benchmarking salary ranges
- Provides an interactive way to explore salary trends

Step 7.3 Dashboard Visualization

To enhance the interpretability of the analysis and model results, an interactive dashboard was developed using Power BI. The dashboard provides a visual representation of salary trends and allows users to explore the data dynamically through filters and interactive charts.

The objective of this dashboard is to present key insights in a simple and intuitive manner, enabling better understanding of salary patterns across different job attributes.

Dashboard Overview

The dashboard consists of multiple visual components that collectively provide a comprehensive view of salary distribution and influencing factors.

At the top of the dashboard, key performance indicators (KPIs) are displayed:

- **Average Salary:** Represents the overall average salary across all job postings
- **Total Jobs:** Indicates the total number of job postings analyzed
- **Maximum Salary:** Shows the highest salary observed in the dataset

These KPIs provide a quick summary of the dataset.

Visualizations Included

The following charts are included in the dashboard:

- **Salary Distribution :** Displays the distribution of average salaries, helping identify common salary ranges and outliers
- **Average Salary by Career Level :** Shows how salary varies across different experience levels
- ****Average Salary by Agency:** Highlights salary differences across organizations (Top agencies)
- **Average Salary by Job Category:** Compares salary across different job domains
- **Salary Trend Over Time:** Illustrates how salary patterns change over months
- **Salary Comparison by Employment Type:** Compares salaries between full-time and part-time roles

Filters and Interactivity

The dashboard includes interactive filters that allow users to explore data dynamically:

- Career Level
- Job Category
- Posting Year (range slider)

These filters enable users to analyze specific segments of the data and gain deeper insights.

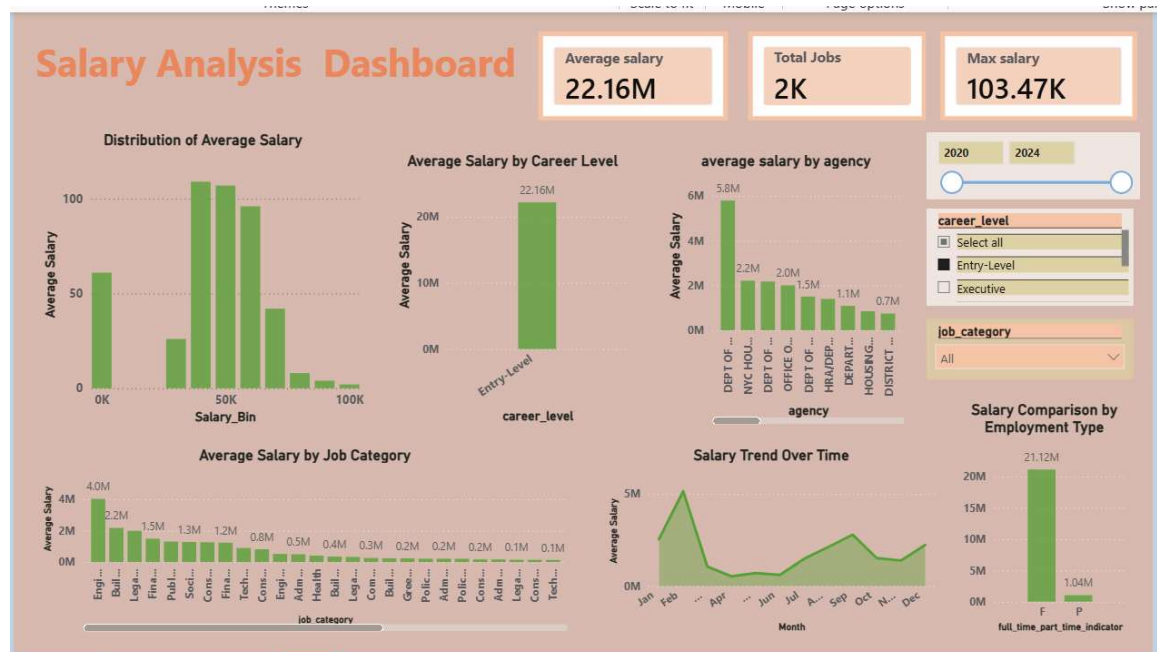
Key Insights from Dashboard

- Salary increases significantly with higher career levels

- Certain job categories consistently offer higher salaries
- Full-time roles provide better compensation compared to part-time roles
- Salary trends vary over time, indicating changes in hiring demand
- Different agencies offer different salary ranges **Business Value**

The dashboard provides a user-friendly interface for analyzing salary trends and supports data-driven decision-making.

- Helps organizations benchmark salary structures
- Assists job seekers in understanding market trends
- Improves interpretability of analytical results



Step8 : Key Insights from Data

1. Career Level Strongly Influences Salary

- One of the most significant observations is that salary increases consistently with career level.
- Entry-level positions have lower salaries Manager and executive roles have significantly higher salaries

👉 Insight: Experience and seniority are major factors in determining salary.

2. Employment Type Impacts Salary

- Full-time roles offer much higher salaries compared to part-time roles

👉 Insight: Employment type plays a critical role in compensation structure.

3. Salary Frequency Matters

- Annual salaries dominate the dataset and show higher values
- Hourly and daily wages are comparatively lower

👉 Insight: Standardizing salary frequency is important for fair comparison and prediction.

4. Job Category Affects Salary Variation

- Different job categories have significantly different salary ranges.
- Technical and managerial roles tend to have higher salaries Administrative or support roles have lower salary ranges

👉 Insight: Domain of work is a key driver of salary differences.

5. Agency-Level Salary Differences

- Different agencies offer varying salary levels for similar roles

👉 Insight: Organizational structure and budget influence compensation.

6. Temporal Trends in Salary

- Salary trends show variation across years and months
- Some periods have higher job postings and slightly higher salaries

👉 Insight: Market demand and hiring cycles influence salary trends.

** Model-Based Insights**

From the model results:

- Random Forest captured complex relationships between features
- Career level, job category, and employment type were among the most influential features
- High R^2 score indicates strong predictive capability

👉 Insight: Salary prediction is influenced by multiple interacting factors rather than a single variable.

Business Recommendations

- For Organizations
 - Use data-driven salary benchmarking to remain competitive
 - Offer better compensation for high-demand roles
 - Align salary with experience level and job category
- For Job Seekers
 - Focus on skill development to move to higher career levels

- Target high-paying job categories
- Prefer full-time roles for better compensation
- For HR & Recruiters
 - Use predictive models to estimate salary ranges
 - Improve hiring strategies using data insights
 - Optimize compensation planning

Step 9: Model Limitations

While the developed model demonstrates strong predictive performance, it is important to acknowledge certain limitations that may affect its real-world applicability. Understanding these limitations helps in interpreting the results more accurately and identifying areas for improvement.

Limited Feature Scope

The model is built using only the available features in the dataset.

- Important real-world factors such as:
- Candidate skills
- Years of experience
- Educational background

are not included.

👉 Impact: The model may not fully capture all factors influencing salary.

Data Quality Constraints

- Some columns contained missing values
- Several text-based columns were removed due to high complexity

👉 Impact: Potentially useful information might have been excluded, which could affect prediction accuracy.

High Cardinality in Categorical Features

- Features like agency and work location contain many unique values

👉 Impact: Encoding such features may introduce noise and reduce model interpretability.

Overfitting Risk in Tree-Based Models

- Models like Decision Tree showed very high accuracy but were prone to overfitting
- Although Random Forest reduces this issue, it may still exist to some extent

👉 Impact: Model performance on unseen data may slightly vary.

Limited Generalization

- The dataset is specific to NYC job postings

👉 Impact: The model may not generalize well to other regions or industries without retraining.

Lack of Real-Time Data

- The dataset represents historical job postings

👉 Impact: The model may not reflect current market trends or sudden changes in salary patterns.

Step 10: Future Improvements

While the current model achieves high accuracy and demonstrates strong predictive performance, there are several opportunities to further enhance the system. These improvements focus on increasing model accuracy, expanding feature coverage, and improving real-world usability.

Incorporating Additional Features

The model can be improved by including more detailed and relevant features such as:

- Years of experience
- Educational qualifications
- Technical skills and certifications
- Industry-specific attributes

👉 Benefit: Provides a more comprehensive understanding of salary determinants and improves prediction accuracy.

Advanced Feature Engineering

- Apply transformations to handle skewness (e.g., log transformation)
- Create interaction features between important variables
- Extract meaningful insights from existing data

👉 Benefit: Helps the model capture complex patterns more effectively.

Utilizing NLP for Text Data

- Currently, text-based columns such as job description and preferred skills were removed.
- These can be processed using Natural Language Processing (NLP) techniques
- Techniques like TF-IDF or word embeddings can be applied

👉 Benefit: Extracts valuable insights from textual data, improving model performance.

Hyperparameter Optimization

- Apply advanced tuning techniques such as GridSearchCV or RandomizedSearchCV
- Experiment with more parameter combinations

👉 Benefit: Further improves model performance and stability.

Model Ensemble Techniques

- Combine predictions from multiple models (e.g., stacking, blending)

👉 Benefit: Improves robustness and overall prediction accuracy.

Real-Time Data Integration

- Integrate live job posting data
- Continuously update the model with new data

👉 Benefit: Ensures predictions remain relevant to current market trends.

◆ 10.7 Deployment Enhancements**

- Deploy the application on cloud platforms (AWS, Azure, or GCP)
- Improve UI/UX of the Streamlit app
- Add features like salary range prediction and comparison

👉 Benefit: Enhances scalability and user experience.

Step 11: Conclusion

This project successfully demonstrates the development of an end-to-end machine learning system for salary prediction using real-world job postings data. The objective of predicting salary based on job-related features was achieved through a structured and systematic approach.

Starting from data preprocessing, the dataset was cleaned and transformed into a usable format. Feature engineering techniques such as creating average salary and extracting temporal features helped in enhancing the dataset. Exploratory Data Analysis provided valuable insights into the factors influencing salary, guiding effective feature selection.

Multiple machine learning models were implemented and evaluated, and the Random Forest Regressor was selected as the final model due to its superior performance and strong generalization capability. The model achieved high accuracy with an R^2 score of approximately 0.996, indicating its effectiveness in capturing salary patterns.

To demonstrate real-world applicability, the model was deployed using a Streamlit application, allowing users to input job details and receive instant salary predictions. This highlights the practical value of the solution in supporting data-driven decision-making.

Overall, this project not only showcases technical expertise in data preprocessing, analysis, and modeling but also emphasizes the importance of translating machine learning solutions into usable applications. It provides a strong foundation for building scalable and impactful data-driven systems in the future.

Project Details

Name: Illuri Sunil Kumar Reddy **Project Title:** Machine Learning-Based Salary Prediction

Tools Used: Python, Pandas, NumPy, Scikit-learn, XGBoost, Streamlit

In []: